

ANU Physics Academic Integrity Declaration Form

Name: chhaya ramnani Student ID: u8336188

Date: 24 sep 2026 Assessment Item: week 7 lab

Collaboration with others

You are encouraged to discuss problems with other students, however you must submit your own work. The minimum penalty for plagiarism is a fail on the assessment task, and all incidents of plagiarism are recorded on a student's academic record.

If you have collaborated with others, please note these people here:

Use of AI Tools

You may not present material produced by generative AI as your own work, as this is an academic integrity breach. If you have used AI in this assessment, the use must be declared here:

☒ **No AI tools were used** for this assignment

☐ **AI tools were used** as described below:

AI Tools Used: _____

How AI was used (check all that apply):

☐ Mathematical calculations ☐ Problem solving ☐ Code generation
☐ Writing assistance ☐ Research ☐ Other: _____

Chat Evidence Required - you must include your complete AI conversation(s):

☐ Full chat transcript pasted below
☐ Full chat screenshots/files attached
☐ Shared link: _____

I declare that:

- ☒ This AI use declaration is complete and accurate
- ☒ I have included unedited records of all AI interactions
- ☒ I understand the physics concepts and can explain my work
- ☒ I remain responsible for all content in my submission

Signature:  Date: 24 sep 2026

Lab Report: Week 7 - Drag Force

Chhaya Ramnani (UID: u8336188)

Lab partners: Tsz Wing Candice Ng, Chinatsu Komada

2026-09-24

Imports and Functions

Python was used throughout. `pandas` for reading Tracker's exported CSVs, `numpy` for the numerical work, `matplotlib` for graphs, `scipy` for ODR and the chi-squared test, and `uncertainties` for propagating uncertainties.

```
from dataclasses import dataclass
import os

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy import odr, stats
from uncertainties import ufloat
```

Tracker doesn't export a per-point uncertainty on t or v_y , so these were assigned by hand:

```
g = 9.7976
FPS = 30
sT = 1 / (2 * FPS)      # timing uncert half a frame
sVy = 0.05              # m/s, velocity uncert from pixel-tracking
```

`odr_fit`, `sigma_test` etc carried over from all other labs. It extracts the nominal values and uncertainties from the `ufloat` data, performs the ODR fit for whatever model it's given, and returns the fitted parameters with their uncertainties

```
@dataclass
class odr_fit_result:
    parameters: np.ndarray
    parameter_errors: np.ndarray
    r_squared: float
    reduced_chi_squared: float
```

```

converged: bool
x_nom: np.ndarray
y_nom: np.ndarray
x_err: np.ndarray
y_err: np.ndarray

def odr_fit(x: list[ufloat], y: list[ufloat], model_func, beta0: list[float]) -
> odr_fit_result:
    x_nom = np.array([v.n for v in x])
    x_err = np.array([v.s for v in x])
    y_nom = np.array([v.n for v in y])
    y_err = np.array([v.s for v in y])

    data = odr.RealData(x_nom, y_nom, sx=x_err, sy=y_err)
    model = odr.Model(model_func)
    odr_result = odr.ODR(data, model, beta0=beta0).run()

    parameters = odr_result.beta
    parameter_errors = odr_result.sd_beta
    converged = odr_result.info in (1, 2, 3)
    if not converged:
        print(f"did not converge (info={odr_result.info})")

    residuals = y_nom - model_func(parameters, x_nom)
    ss_res = np.sum(residuals**2)
    ss_tot = np.sum((y_nom - np.mean(y_nom)) ** 2)
    r_squared = 1 - ss_res / ss_tot if ss_tot != 0 else np.nan

    return odr_fit_result(
        parameters=parameters,
        parameter_errors=parameter_errors,
        r_squared=r_squared,
        reduced_chi_squared=odr_result.res_var,
        converged=converged,
        x_nom=x_nom,
        y_nom=y_nom,
        x_err=x_err,
        y_err=y_err,
    )

```

```

def chi_squared_test(chi2_stat, dof):
    p_value = stats.chi2.sf(chi2_stat, dof)
    print(f"chi-squared: {chi2_stat:.3f} (dof = {dof})")
    print(f"reduced chi-squared: {chi2_stat/dof:.3f}")
    print(f"p-value: {p_value:.4f}")
    if p_value > 0.05:
        print("results AGREE with the model (p > 0.05)")
    else:
        print("results DISAGREE with the model (p <= 0.05)")
    return p_value

def sigma_test(k1, sk1, k2, sk2, label="parameter"):
    sigma_diff = abs(k1 - k2) / np.sqrt(sk1**2 + sk2**2)

    print(f"\nSIGMA TEST ({label})")
    print(f"measured value      : {k1:.3f} +/- {sk1:.3f}")
    print(f"theoretical value : {k2:.3f} +/- {sk2:.3f}")
    print(f"discrepancy          : {sigma_diff:.2f} sigma")

    if sigma_diff <= 3.0:
        print("results AGREE within experimental uncertainty")
    else:
        print("results DISAGREE")

    return sigma_diff

```

The two models. `no_drag` is Eq (1) with the slope fixed at $-g$ (only v_0 is free). `drag` is Eq (4), free in both v_0 and v_T . The lab notes' Eq (4) has v and v_T positive (downward speed), but our y -axis points up so falling velocities are negative. Rewriting it for that convention means $v_T < 0$ and a sign flip on the gt/v_T term, which is what is done below.

```

def no_drag_model(p, t):
    return p[0] - g * t

def drag_model(p, t):
    v0, vT = p
    ratio = np.clip(v0 / vT, -0.999999, 0.999999) # blows up at +/-1
    return vT * np.tanh(-g * t / vT + np.arctanh(ratio))

```

Tracker already gives a v_y column in the export, so that's what gets used directly. It leaves the first and last frames empty, so those rows are dropped, and t is shifted so $t = 0$ is the first tracked frame.

```

def load_data(path):
    df = pd.read_csv(path).rename(columns={"v_y": "vy"})
    df["t"] = df["t"] - df["t"].iloc[0]
    df = df.dropna(subset=["vy"])

    t = [ufloat(val, sT) for val in df["t"]]
    vy = [ufloat(val, sVy) for val in df["vy"]]
    return t, vy

```

plotting helpers: one for the fit, one for the residuals. The residuals have two panels because the no-drag ones are about 100x bigger than the drag ones, so on a shared axis the drag residuals look like a flat line.

```

def plot_fit(fit_result, model_func, title, filename):
    x, y = fit_result.x_nom, fit_result.y_nom
    x_fit = np.linspace(min(x), max(x), 200)
    y_fit = model_func(fit_result.parameters, x_fit)

    plt.figure(figsize=(7, 5))
    plt.errorbar(x, y, xerr=fit_result.x_err, yerr=fit_result.y_err, fmt="o",
        capsize=3,
        color="navy", ecolor="crimson", label="Tracker Data")
    plt.plot(x_fit, y_fit, "k--", label=f"Drag fit ( $R^2 = {fit_result.r_squared:.3f}$ )")
    plt.xlabel("Time  $t$  (s)")
    plt.ylabel("Velocity  $v_y$  (m/s)")
    plt.title(title)
    plt.grid(True, alpha=0.6)
    plt.legend()
    plt.tight_layout()
    plt.savefig(filename, dpi=300)
    plt.show()

def plot_residuals(fit_nodrag, fit_drag, title, filename):
    x, y = fit_drag.x_nom, fit_drag.y_nom
    resid_nodrag = y - no_drag_model(fit_nodrag.parameters, x)
    resid_drag = y - drag_model(fit_drag.parameters, x)

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7, 6), sharex=True)
    for ax, resid, colour, marker, name in [
        (ax1, resid_nodrag, "darkorange", "^", "No-drag residuals (Eq. 1)",
        (ax2, resid_drag, "navy", "o", "Drag residuals (Eq. 4)",

```

```

]:
    ax.axhline(0, color="black", linestyle="--")
    ax.scatter(x, resid, color=colour, marker=marker, label=name)
    ax.set_ylabel("residual (m/s)")
    ax.grid(True, alpha=0.6)
    ax.legend()
ax2.set_xlabel("t (s)")
ax1.set_title(title)
fig.tight_layout()
fig.savefig(filename, dpi=300)
plt.show()

```

One function per video: load, fit no-drag and drag, run the chi-squared check on both, make both plots.

```

def analysis(label, csv_path, graph_dir):
    t_data, vy_data = load_data(csv_path)
    vy = np.array([v.n for v in vy_data])

    fit_nodrag = odr_fit(t_data, vy_data, no_drag_model, beta0=[vy[0]])
    fit_drag = odr_fit(t_data, vy_data, drag_model, beta0=[vy[0], vy.min() * 1.2])

    v0_nodrag = ufloat(fit_nodrag.parameters[0], fit_nodrag.parameter_errors[0])
    v0_drag = ufloat(fit_drag.parameters[0], fit_drag.parameter_errors[0])
    vT = ufloat(fit_drag.parameters[1], fit_drag.parameter_errors[1])

    print(f"\n{label}")
    print(f"no-drag fit (Eq. 1): v0 = {v0_nodrag:.3f} m/s | R^2 = {fit_nodrag.r_squared:.4f}")
    print(f"drag fit (Eq. 4): v0 = {v0_drag:.3f} m/s, vT = {vT:.3f} m/s | R^2 = {fit_drag.r_squared:.4f}")

    n = len(vy)
    print("\nnno-drag chi-squared")
    chi_squared_test(fit_nodrag.reduced_chi_squared * (n - 1), n - 1)
    print("\ndrag chi-squared")
    chi_squared_test(fit_drag.reduced_chi_squared * (n - 2), n - 2)

    name = label.lower().replace(" ", "_")
    plot_fit(fit_drag, drag_model, f"{label}: Drag Fit", f"{graph_dir}/{name}_drag_fit.png")
    plot_residuals(fit_nodrag, fit_drag, f"{label}: Residuals", f"{graph_dir}/{name}_residuals.png")

```

```
return vT
```

Pre-Lab Questions

Question 1

- (1) that is the calibration button. it lets us drag a stick over a known length in a video and tell Tracker that distance, which is how Tracker converts pixel distances into metres
- (2) that is the coordinate axes button. it lets us set the origin and orientation of the x-y axes on our video and rotate the axes if our camera wasn't perfectly vertical.

Question 2

From Eq. (3) in the handout:

$$mg = Cv_T^2$$
$$v_T = \sqrt{\frac{mg}{C}}$$

For objects with the same shape and size, the cross-sectional area is the same, so C is constant.

$$v_T \propto \sqrt{m}$$

the terminal velocity increases with the square root of the mass. for example, if the mass is doubled, the terminal velocity should increase by a factor of

$$\sqrt{2} \approx 1.41.$$

so, for the two nested patty pans, which have ~twice the mass of a single patty pan but a similar cross-sectional area,

$$\frac{v_{T,\text{double}}}{v_{T,\text{single}}} = \sqrt{2}$$

Aim

The aim of this experiment is to investigate the effect of air resistance on a falling patty pan by measuring its vertical velocity as it falls. The experimental data will be compared with both a no-drag model and a model including air resistance. The terminal velocity of a single patty pan will then be compared with that of two nested patty pans.

Theory

An object falling in vacuum has

$$v(t) = v_0 - gt$$

With a drag force $F_{drag} = Cv^2$ opposing the motion, the object reaches a terminal velocity v_T where $mg = Cv_T^2$, and its velocity is

$$v(t) = v_T \tanh \left[\frac{gt}{v_T} + \tanh^{-1} \left(\frac{v_0}{v_T} \right) \right]$$

This is what's in the lab notes- where v and v_T are downward speeds. With the y -axis pointing up (according to the axes set in Tracker, so falling velocities are negative and $v_T < 0$) it becomes

$$v(t) = v_T \tanh \left[-\frac{gt}{v_T} + \tanh^{-1} \left(\frac{v_0}{v_T} \right) \right]$$

Two nested patty pans have essentially the same cross-section to the air as one, so C should stay roughly fixed while the mass doubles. From $mg = Cv_T^2$,

$$\frac{v_T(2 \text{ pans})}{v_T(1 \text{ pan})} = \sqrt{\frac{2m}{m}} = \sqrt{2} \approx 1.414$$

which is the number we use to check against the data in **Results**.

Procedure

A patty pan was dropped close to a wall with a 1m ruler taped against it, filmed with a camera held vertical about 2.5 m away, at a height roughly halfway up the fall. This was done for a single patty pan and repeated for two patty pans nested together.

Each video was opened in Tracker, calibrated against the tape marks, with the origin set at the object's position in the first frame of the fall. The object was tracked frame by frame as a point mass, and the data exported to two csv files (t , x , y and Tracker's own v_y). Tracker doesn't give a v_y value for the first or last frame (it computes velocity from the frames either side), so those two rows were dropped from each file, leaving 47 points for the single pan and 39 for the two pans. t was shifted so $t = 0$ is the first tracked frame, and v_0 was left as a free fit parameter.

Quantity	Uncertainty
Time t	half a frame (1/2 FPS)
Velocity v_y	0.05 m/s (pixel-tracking resolution)

Both models were fitted with `scipy.odr` rather than ordinary least squares, since both t and v_y carry uncertainty.

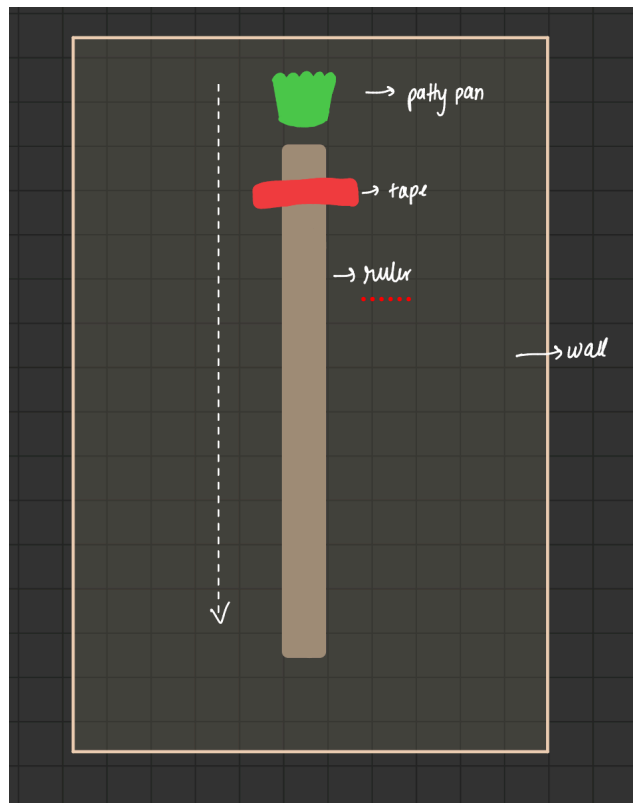


Figure 1: Diagram of the setup

Results

Single Patty Pan

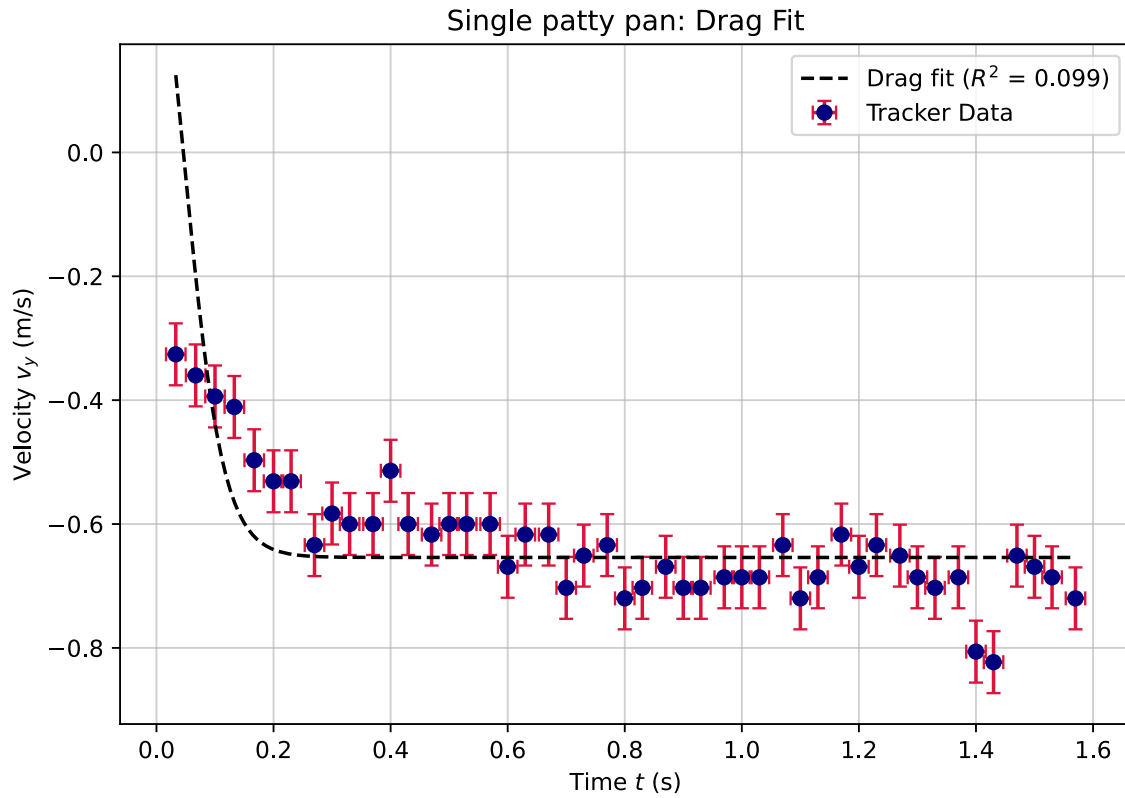
```
graph_dir = "graphs"
os.makedirs(graph_dir, exist_ok=True)

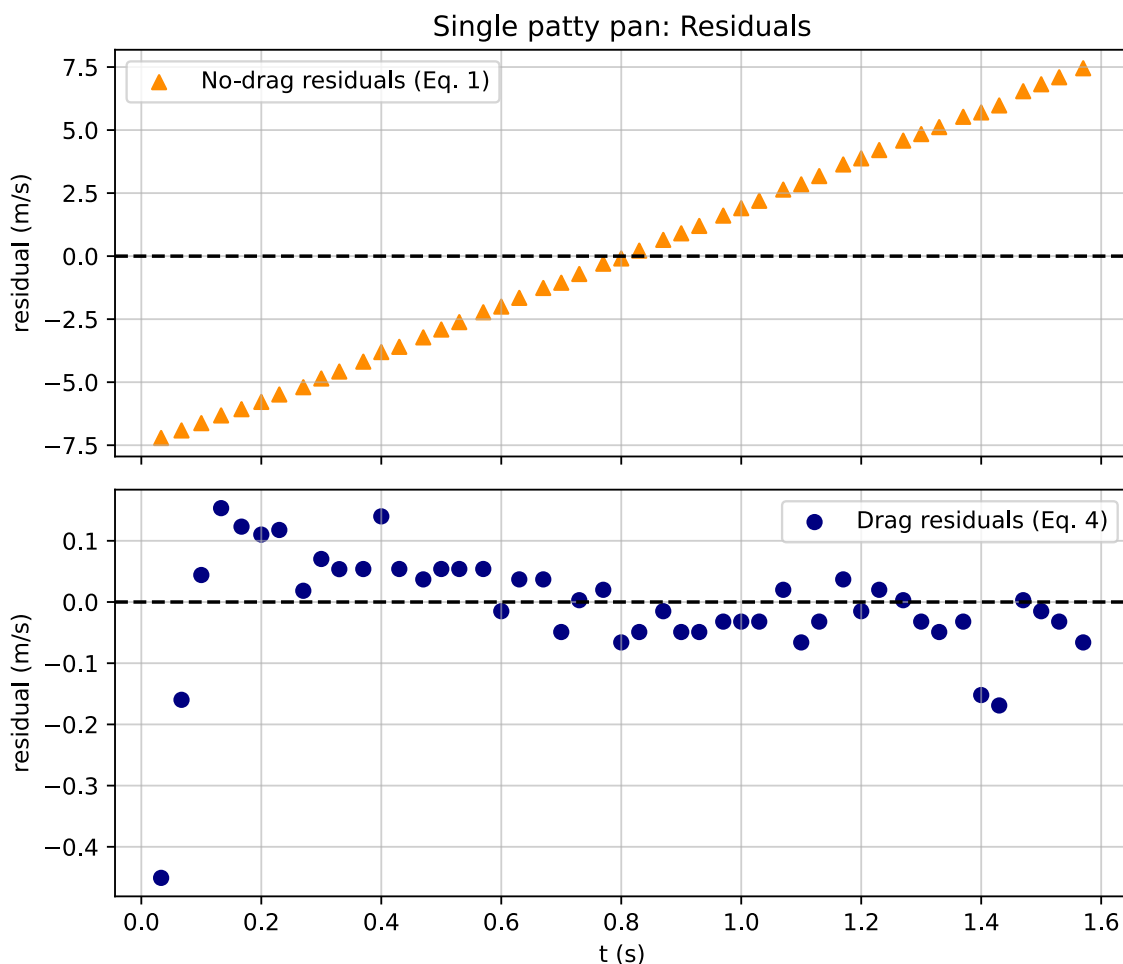
vT_1 = analysis("Single patty pan", "files/one_pan.csv", graph_dir)
```

```
Single patty pan
no-drag fit (Eq. 1): v0 = 7.211+/-0.642 m/s | R^2 = -1868.8645
drag fit (Eq. 4): v0 = 0.390+/-0.076 m/s, vT = -0.654+/-0.010 m/s | R^2 = 0.0988

no-drag chi-squared
chi-squared: 30535.599 (dof = 46)
reduced chi-squared: 663.817
p-value: 0.0000
results DISAGREE with the model (p <= 0.05)
```

```
drag chi-squared  
chi-squared: 81.991 (dof = 45)  
reduced chi-squared: 1.822  
p-value: 0.0006  
results DISAGREE with the model (p <= 0.05)
```





For the single patty pan, the no-drag model is clearly inconsistent with the data. We got $R^2 \approx -1869$, meaning the model performs much worse than simply using the average velocity as a prediction. Equation 1 assumes that the pan continues accelerating at 9.8 m/s^2 , whereas the measured velocity becomes approximately constant at around -0.65 m/s from about $t = 0.3 \text{ s}$ onwards. The fitted value of v_0 was approximately $+7.2 \text{ m/s}$ (the pan would have to move upwards!). The residual plot shows that the residuals form an almost straight trend from about -7 to $+7.5 \text{ m/s}$, showing that the model has the wrong overall slope rather than simply being slightly offset.

The drag model in Equation 4 gives

$$v_T = -0.654 \pm 0.010 \text{ m/s} .$$

This model follows the data much more closely. The residuals remain within roughly $\pm 0.15 \text{ m/s}$ for most of the fall, with the largest differences occurring in the first two points, at approximately -0.45 m/s and -0.16 m/s . There is also a small pattern in the residuals- they are slightly above

zero until around $t = 0.6$ s, then mostly below zero afterwards. The two lower residuals near $t = 1.4$ s also correspond well with the swaying motion we observed while tracking the pan.

The fit is not statistically perfect. The reduced chi-squared value is approximately 1.8, giving $p \approx 0.0006$ using the assigned $\sigma_{v_y} = 0.05$ m/s. This means the model would be rejected at the 5% significance level under the assumptions of the chi-squared test. However, the residuals are still much smaller and less structured than for the no-drag model. R^2 is only about 0.10 because the velocity is almost constant, so the mean is already a good predictor.

Two nested patty pans

```
vT_2 = analysis("Two nested patty pans", "files/two_pan.csv", graph_dir)
```

```
Two nested patty pans
```

```
no-drag fit (Eq. 1):  $v_0 = 5.818 \pm 0.565$  m/s |  $R^2 = -283.6400$ 
```

```
drag fit (Eq. 4):  $v_0 = 0.668 \pm 0.069$  m/s,  $v_T = -0.825 \pm 0.026$  m/s |  $R^2 = -0.1411$ 
```

```
no-drag chi-squared
```

```
chi-squared: 16237.937 (dof = 38)
```

```
reduced chi-squared: 427.314
```

```
p-value: 0.0000
```

```
results DISAGREE with the model (p <= 0.05)
```

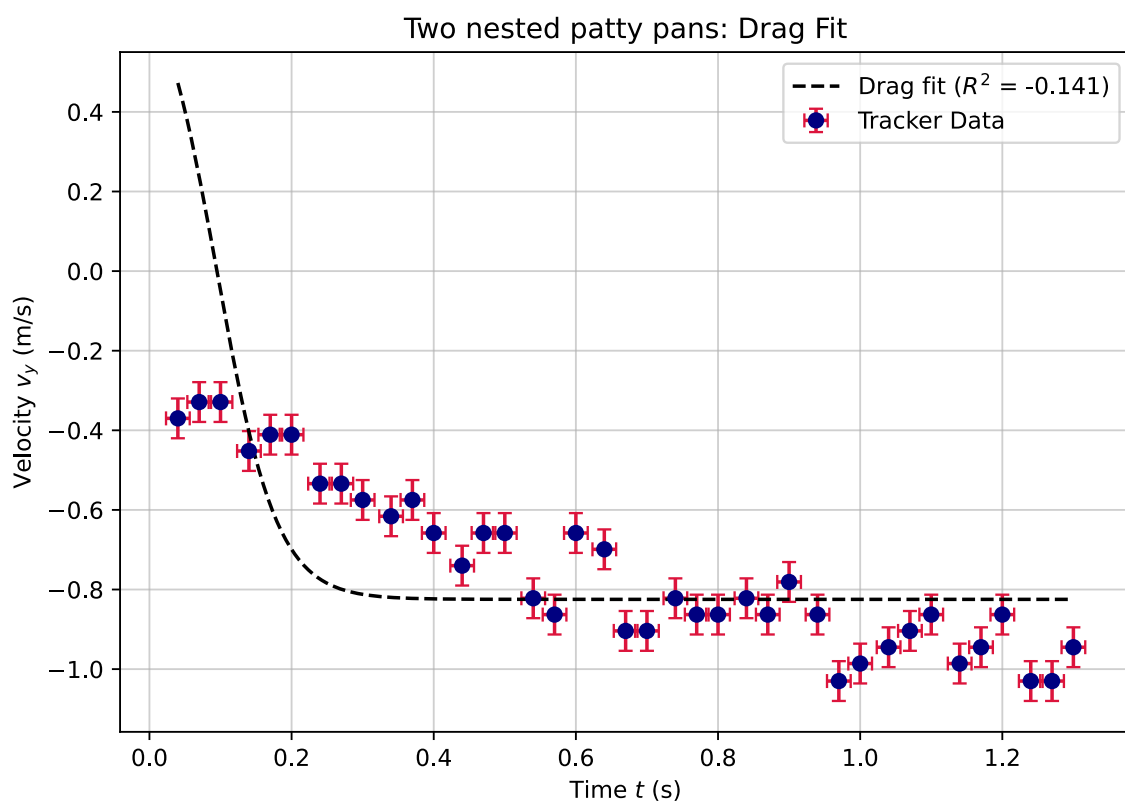
```
drag chi-squared
```

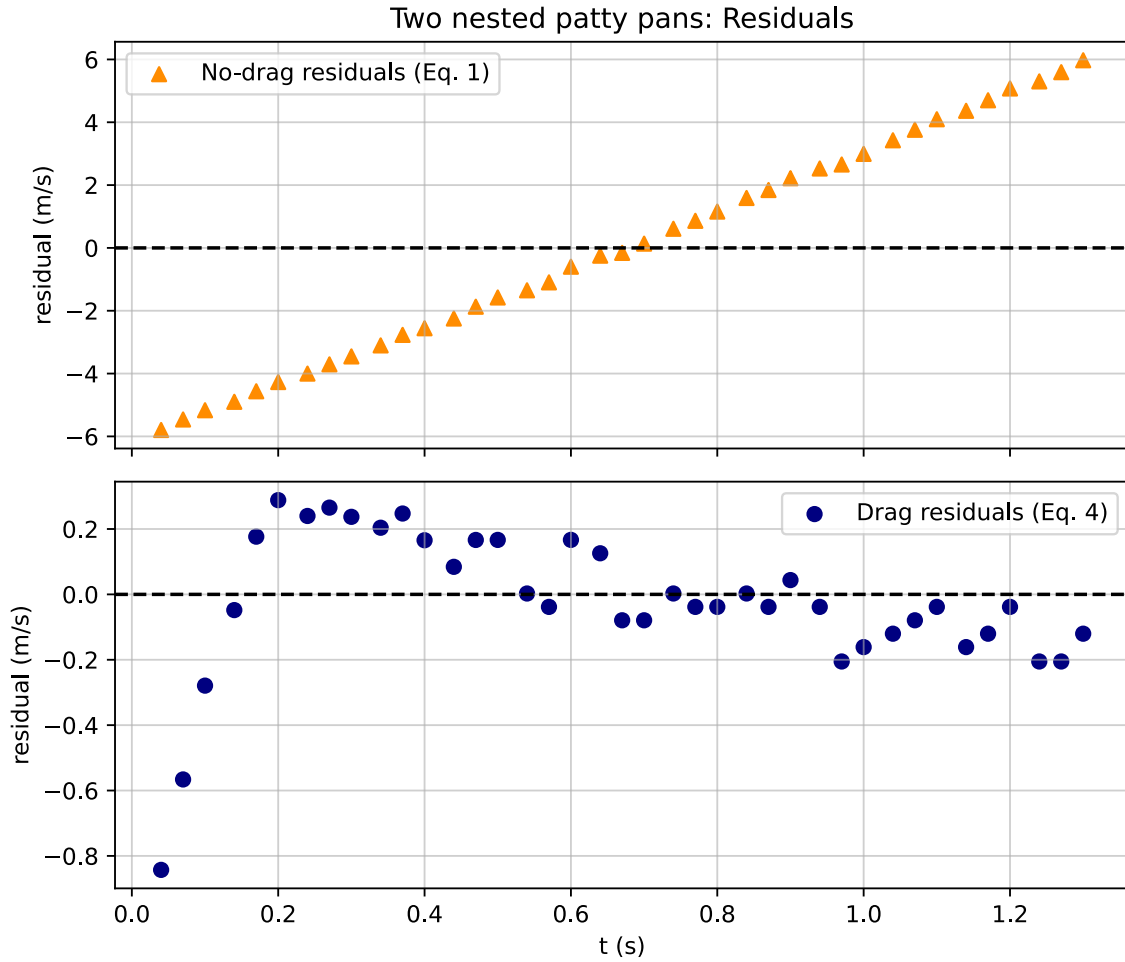
```
chi-squared: 317.385 (dof = 37)
```

```
reduced chi-squared: 8.578
```

```
p-value: 0.0000
```

```
results DISAGREE with the model (p <= 0.05)
```





Same pattern with the no-drag model: $R^2 \approx -284$, fitted $v_0 \approx +5.8$ m/s, and a straight-line residual from about -6 to $+6$ m/s. It fails for the same reason as before.

The drag fit is where this pan looks different. v_y goes from about -0.37 m/s at the start, gets to around -0.85 m/s by $t \approx 0.55$ s, then keeps drifting down to about -1.0 m/s by $t \approx 1$ s. The fit puts its plateau at $v_T = -0.825 \pm 0.026$ m/s, which is too low early on and too high late on, and as seen on the residual plots, a big miss on the first few points (down to about -0.84 m/s), positive up to $t \approx 0.6$ s, then mostly negative after $t \approx 0.9$ s. That's a systematic pattern, and the fit is bad on the stats too - reduced $\chi^2 \approx 8.6$ and $R^2 \approx -0.14$.

The two-pan fall takes longer to settle than Eq. 4 allows (about $v_T/g \approx 0.08$ s), and it may not have fully settled by the time the footage ends. So $v_T = -0.825$ m/s is probably an underestimate of the true terminal velocity.

Comparing terminal velocities

```
ratio = vT_2 / vT_1
print(f"vT (single) = {vT_1:.3f} m/s")
print(f"vT (double) = {vT_2:.3f} m/s")
print(f"measured vT ratio = {ratio:.3f}")

_ = sigma_test(ratio.n, ratio.s, np.sqrt(2), 0.0, label="vT ratio vs sqrt(2)")
```

```
vT (single) = -0.654+/-0.010 m/s
vT (double) = -0.825+/-0.026 m/s
measured vT ratio = 1.261+/-0.045
```

```
SIGMA TEST (vT ratio vs sqrt(2))
measured value      : 1.261 +/- 0.045
theoretical value   : 1.414 +/- 0.000
discrepancy         : 3.40 sigma
results DISAGREE
```

This is the main point of the whole experiment. Prelab problem 2 solved gives that doubling the mass by nesting the pans should make v_T bigger by a factor of $\sqrt{2} \approx 1.414$. What we actually got was a ratio of 1.261 ± 0.045 , which sits 3.40σ away from that prediction.

That's just past the usual 3σ agreement cutoff, so it's a disagreement though not a dramatic one. However, the way the two-pan data were fitted is important here. The two-pan velocity does not show a clear plateau before the end of the video, and the residuals show a clear systematic pattern. If the fitted v_T is below the true terminal velocity because the pan was still accelerating, this would reduce the measured ratio.

$$\frac{v_T(2 \text{ pans})}{v_T(1 \text{ pan})} = 1.261 \pm 0.045 \quad \text{vs predicted } \sqrt{2} \approx 1.414 \quad (3.40\sigma)$$

Limitations & Possible Sources of Discrepancy

1. The patty pan doesn't fall perfectly symmetrically. it can wobble, tumble, or tilt on the way down instead of holding a constant cross-section to the air, which would make the drag coefficient C drift during the drop rather than stay fixed like Eq. 4 assumes. We saw a this while tracking: both videos' $x(t)$ traces drift sideways which corresponds with the pans being light enough that air currents or a not-exactly-vertical release could nudge them around.

2. The data relaxes to terminal velocity slower than Eq. 4 allows. For both pans, Eq. 4 predicts terminal velocity gets reached in something like 0.07-0.08 s, but the data takes about 0.25 s (single) and 0.5 s or more (two pans). That mismatch is why the fitted v_0 comes out positive and why the early residuals are the biggest ones in both fits. It could be the pan changing orientation as it settles $\Rightarrow C$ isn't constant early on, or the release adding some initial motion.

3. Tracker's v_y points aren't independent. Tracker works out v_y from the positions on either side of each frame, so neighbouring v_y values share position data.

4. Click precision in Tracker. v_y comes from finite differences of manually-clicked positions, so any wobble in exactly where the pan's centre got clicked turns into velocity noise. The $\sigma_{v_y} = 0.05$ m/s used throughout is a rough estimate of that, not something we actually measured frame by frame and the chi-squared results suggest it's on the low side.

5. Cross-section assumption for the mass-scaling test. The $\sqrt{2}$ prediction only holds if C stays the same between one and two nested pans. If nesting them changes the shape they present to the air even a little, that shows up in the sigma test as a "discrepancy", which isn't entirely correct.

Conclusion

The no-drag model does not describe either dataset well- $R^2 \ll 0$, $p \approx 0$, and it needs an upward v_0 of several m/s to come close to theory- so air resistance is clearly a main component. The drag model does better and converges for both videos, giving $v_T(1 \text{ pan}) = -0.654 \pm 0.010$ m/s and $v_T(2 \text{ pans}) = -0.825 \pm 0.026$ m/s, but it isn't a statistically good fit for either. It's decent for the single pan (reduced $\chi^2 \approx 1.8$) and poor for the two pans (reduced $\chi^2 \approx 8.6$), where the pans were still speeding up when the footage ended.

That gives a ratio of 1.261 ± 0.045 against the $\sqrt{2} \approx 1.414$ prediction. That's a 3.40σ gap, so it doesn't agree within our 3σ cutoff. However, the two-pan v_T is likely affected by the fact that the pan had not fully settled into terminal motion. The assumption that the drag coefficient remains unchanged when the pans are nested may also not be completely valid.

If we were to repeat the experiment, we would record a longer fall, or start the pans from a greater height, so that the two-pan velocity had enough time to reach a clear plateau- this would give a more direct measurement of v_T .